

2.1 Cart-Pole Swing-Up with Limited Actuation

2.1(a)

We want the constraint $s_{k+1} = f_d(s_k, u_k)$ to have the form $s_{k+1} = A_k s_k + B_k u_k + c_k$ to derive the convex approximation of the OCP at iteration i , around $(s_k^{(i)}, u_k^{(i)})$.

The linearization of $f_d(s_k, u_k)$ becomes:

$$f_d(s_k^{(i)}, u_k^{(i)}) = f(s_k^{(i)}, u_k^{(i)}) + \frac{\partial f_d}{\partial s}(s_k^{(i)}, u_k^{(i)})(s_k - s_k^{(i)}) + \frac{\partial f_d}{\partial u}(s_k^{(i)}, u_k^{(i)})(u_k - u_k^{(i)})$$

After expanding terms, we come up with

$$A_k = \frac{\partial f_d}{\partial s}(s_k^{(i)}, u_k^{(i)}), \quad B_k = \frac{\partial f_d}{\partial u}(s_k^{(i)}, u_k^{(i)}),$$

$$c_k = f_d(s_k^{(i)}, u_k^{(i)}) - A_k s_k^{(i)} - B_k u_k^{(i)}.$$

The convex subproblem is

$$\begin{aligned} \min_{s, u} \quad & \sum_{k=0}^{N-1} ((s_k - s_{\text{goal}})^T Q (s_k - s_{\text{goal}}) + u_k^T R u_k) + (s_N - s_{\text{goal}})^T P (s_N - s_{\text{goal}}) \\ \text{s.t.} \quad & s_0 = \bar{s}_0, \\ & s_{k+1} = A_k s_k + B_k u_k + c_k, \\ & -u_{\max} \leq u_k \leq u_{\max}, \\ & \|s_k - s_k^{(i)}\|_{\infty} \leq \rho, \quad \|u_k - u_k^{(i)}\|_{\infty} \leq \rho. \end{aligned}$$

2.1(b)

```

1 A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
2 c = f(s, u) - A @ s - B @ u

```

2.1(d)

First use SCP to compute a feasible swing-up trajectory. Once the pendulum is near $(0, \pi, 0, 0)$, switch to a local LQR controller designed around $(0, \pi, 0, 0)$ for stabilization.

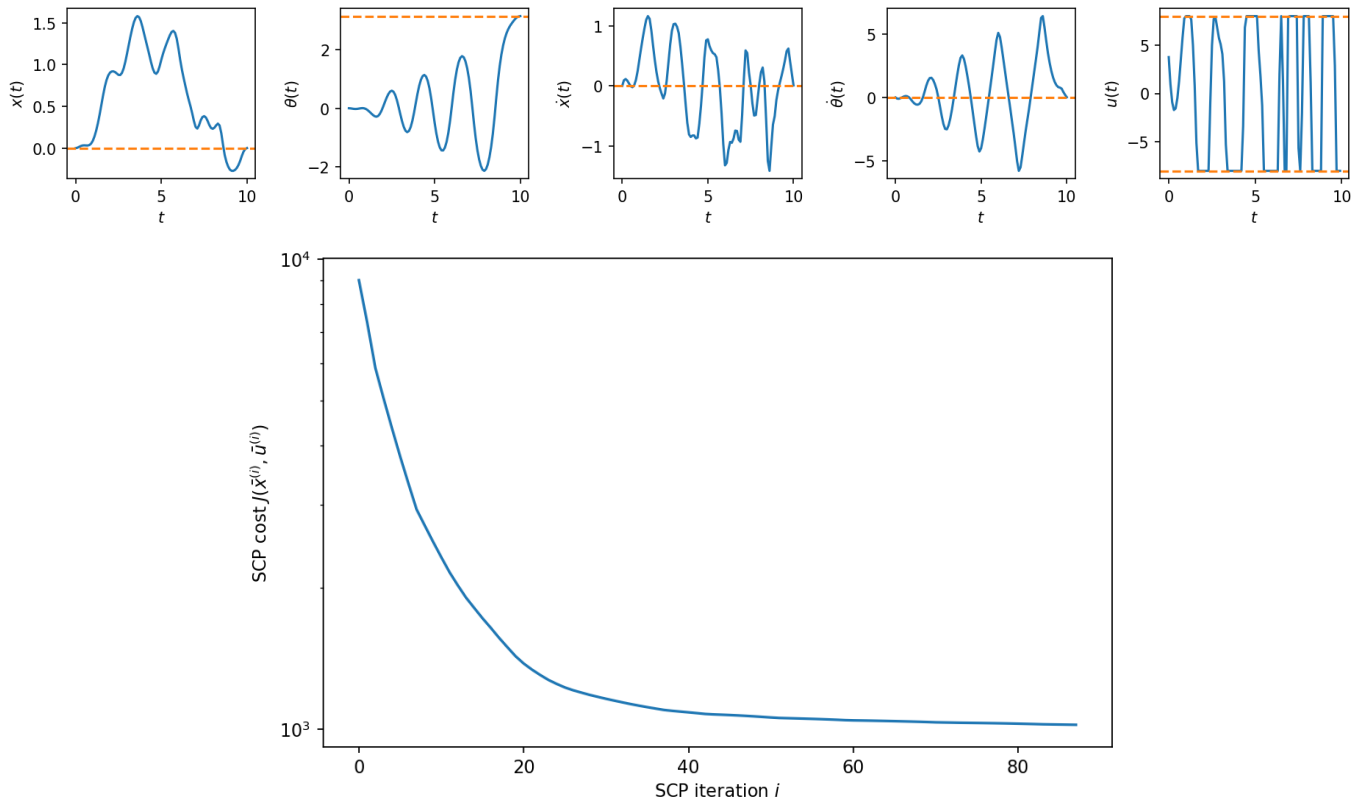


Figure 1: SCP swing-up trajectory with bounded input and trust-region updates.

Code

Code in Appendix: `cartpole_swingup_constrained.py`.

2.2 Shortest Path Through a Grid

2.2(a)

Using dynamic programming from right to left gives the value function

node	$V(\text{node})$
B	0
u_5	10
d_5	11
u_4	18
m_4	16
d_4	20
u_3	25
um_3	23
dm_3	21
d_3	27
u_2	29
m_2	29
d_2	31
u_1	35
d_1	36
A	40

Thus the shortest path has cost

$$\boxed{40}.$$

One optimal path is

$$A \rightarrow u_1 \rightarrow u_2 \rightarrow um_3 \rightarrow u_4 \rightarrow u_5 \rightarrow B.$$

2.2(b)

For an n -segment grid, each route consists of n up moves and n down moves, so exhaustive search evaluates

$$\boxed{\binom{2n}{n}}$$

routes. Dynamic programming evaluates each nonterminal node. The number of such DP evaluations is

$$\boxed{n(n+2)}.$$

For $n = 3$, these give $\binom{6}{3} = 20$ and $3(5) = 15$, matching the problem statement.

2.3 Cart-Pole Balance

2.3(a)

With $\tilde{s}_k = s_k - \bar{s}$, Euler discretization gives

$$\tilde{s}_{k+1} \approx A\tilde{s}_k + Bu_k,$$

where

$$A = I + \Delta t \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{m_p g}{m_c} & 0 & 0 \\ 0 & \frac{(m_c + m_p)g}{m_c \ell} & 0 & 0 \end{bmatrix}, \quad B = \Delta t \begin{bmatrix} 0 \\ 0 \\ \frac{1}{m_c} \\ \frac{1}{m_c \ell} \end{bmatrix}.$$

2.3(b)

Using Riccati recursion until $\|P_k - P_{k-1}\|_{\max} < 10^{-4}$ gives

$$K_{\infty} = \begin{bmatrix} 0.73 & -231.85 & 4.22 & -68.25 \end{bmatrix}.$$

2.3(d)

- i. We can reuse A and B because the dynamics Jacobians along the reference do not depend on x , and the desired trajectory keeps $\theta = \pi$, $\dot{\theta} = 0$. Therefore the same upright linearization is valid along the reference.
- iii. The reference cart motion has nonzero acceleration, but the reference does not include a dynamically consistent feedforward force. The pendulum must tilt to generate the horizontal cart acceleration, so keeping $\theta(t) \equiv \pi$ while moving the cart sinusoidally is not a dynamically consistent upright trajectory without feedforward compensation.

2.4 Cart-Pole Swing-Up

2.4(a)

The requested one-line JAX linearization is

```
1 A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
```

2.4(b)

Let

$$\tilde{s}_k = s_k - \bar{s}_k, \quad \tilde{u}_k = u_k - \bar{u}_k.$$

Expanding the quadratic cost gives linear terms

$$q_N = Q_N(\bar{s}_N - s_{\text{goal}}), \quad q_k = Q(\bar{s}_k - s_{\text{goal}}), \quad r_k = R\bar{u}_k.$$

2.4(c)

The iLQR implementation is in `figures/cartpole.swingup.py`. It computes linearizations, performs the Riccati backward pass for feedback gains Y_k and offsets y_k , updates the nominal trajectory, and uses backtracking to keep the update stable.

2.5 Machine Maintenance

Let $V_R(h)$ and $V_B(h)$ denote the optimal expected profit with h weeks remaining when the machine starts the week running or broken.

$$\begin{aligned} V_R(h) &= \max\{40 + 0.6V_R(h-1) + 0.4V_B(h-1), \\ &\quad 30 + 0.3V_R(h-1) + 0.7V_B(h-1)\}, \\ V_B(h) &= \max\{20 + 0.6V_R(h-1) + 0.4V_B(h-1), -50 + V_R(h-1)\}. \end{aligned}$$

With $V_R(0) = V_B(0) = 0$, the computed values are

h	0	1	2	3
$V_R(h)$	0	40	72	104
$V_B(h)$	0	20	52	84

The optimal policy is to perform preventive maintenance when running and to repair when broken. Since the machine is new at week 1 and guaranteed to run through that week, the total expected profit over four weeks is

$$\boxed{100 + V_R(3) = 204}.$$

Code Appendix

Homework part	Code file
2.1(b), 2.1(c), 2.1(d)	cartpole_swingup_constrained.py

cartpole_swingup_constrained.py

Answers 2.1(b), 2.1(c), and 2.1(d).

```
1 """
2 Starter code for the problem "Cart-pole swing-up with limited actuation".
3
4 Autonomous Systems Lab (ASL), Stanford University
5 """
6
7 from functools import partial
8 from pathlib import Path
9
10 from animations import animate_cartpole
11
12 import cvxpy as cvx
13
14 import jax
15 import jax.numpy as jnp
16
17 import matplotlib.pyplot as plt
18
19 import numpy as np
20
21 from tqdm import tqdm
22
23
24 FIG_DIR = Path(__file__).resolve().parents[1] / "latex" / "figures"
25 FIG_DIR.mkdir(parents=True, exist_ok=True)
26
27
28 @partial(jax.jit, static_argnums=(0,))
29 @partial(jax.vmap, in_axes=(None, 0, 0))
30 def affinize(f, s, u):
31     """Affinize the function 'f(s,u)' around '(s,u)'.
```

```

53 # PART (b) #####
54 # INSTRUCTIONS: Use JAX to affimize 'f' around '(s, u)' in two lines.
55 A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
56 c = f(s, u) - A @ s - B @ u
57 # END PART (b) #####
58 return A, B, c
59
60
61 def solve_swingup_scp(f, s0, s_goal, N, P, Q, R, u_max,  $\rho$ , eps, max_iters):
62     """Solve the cart-pole swing-up problem via SCP.
63
64     Arguments
65     -----
66     f: callable
67         A function describing the discrete-time dynamics, such that
68         's[k+1] = f(s[k], u[k])'.
69     s0: numpy.ndarray
70         The initial state (1-D).
71     s_goal: numpy.ndarray
72         The goal state (1-D).
73     N: int
74         The time horizon of the LQR cost function.
75     P: numpy.ndarray
76         The terminal state cost matrix (2-D).
77     Q: numpy.ndarray
78         The state stage cost matrix (2-D).
79     R: numpy.ndarray
80         The control stage cost matrix (2-D).
81     u_max: float
82         The bound defining the control set '[-u_max, u_max]'.
83      $\rho$ : float
84         Trust region radius.
85     eps: float
86         Termination threshold for SCP.
87     max_iters: int
88         Maximum number of SCP iterations.
89
90     Returns
91     -----
92     s: numpy.ndarray
93         A 2-D array where 's[k]' is the open-loop state at time step 'k',
94         for 'k=0, 1, ..., N-1'.
95     u: numpy.ndarray
96         A 2-D array where 'u[k]' is the open-loop state at time step 'k',
97         for 'k=0, 1, ..., N-1'.
98     J: numpy.ndarray
99         A 1-D array where 'J[i]' is the SCP sub-problem cost after the i-th
100         iteration, for 'i=0, 1, ..., (iteration when convergence occurred)'.
101     """
102     n = Q.shape[0] # state dimension
103     m = R.shape[0] # control dimension
104
105     # Initialize dynamically feasible nominal trajectories
106     u = np.zeros((N, m))
107     s = np.zeros((N + 1, n))
108     s[0] = s0
109     for k in range(N):
110         s[k + 1] = fd(s[k], u[k])
111
112     # Do SCP until convergence or maximum number of iterations is reached
113     converged = False
114     J = np.zeros(max_iters + 1)
115     J[0] = np.inf

```

```

116     for i in (prog_bar := tqdm(range(max_iters))):
117         s, u, J[i + 1] = scp_iteration(f, s0, s_goal, s, u, N, P, Q, R, u_max,  $\rho$ )
118         dJ = np.abs(J[i + 1] - J[i])
119         prog_bar.set_postfix({"objective_change": "{:.5f}".format(dJ)})
120         if dJ < eps:
121             converged = True
122             print("SCP converged after {} iterations.".format(i))
123             break
124         if not converged:
125             raise RuntimeError("SCP did not converge!")
126     J = J[1 : i + 1]
127     return s, u, J
128
129
130 def scp_iteration(f, s0, s_goal, s_prev, u_prev, N, P, Q, R, u_max,  $\rho$ ):
131     """Solve a single SCP sub-problem for the cart-pole swing-up problem.
132
133     Arguments
134     -----
135     f: callable
136         A function describing the discrete-time dynamics, such that
137         's[k+1] = f(s[k], u[k])'.
138     s0: numpy.ndarray
139         The initial state (1-D).
140     s_goal: numpy.ndarray
141         The goal state (1-D).
142     s_prev: numpy.ndarray
143         The state trajectory around which the problem is convexified (2-D).
144     u_prev: numpy.ndarray
145         The control trajectory around which the problem is convexified (2-D).
146     N: int
147         The time horizon of the LQR cost function.
148     P: numpy.ndarray
149         The terminal state cost matrix (2-D).
150     Q: numpy.ndarray
151         The state stage cost matrix (2-D).
152     R: numpy.ndarray
153         The control stage cost matrix (2-D).
154     u_max: float
155         The bound defining the control set '[-u_max, u_max]'.
156      $\rho$ : float
157         Trust region radius.
158
159     Returns
160     -----
161     s: numpy.ndarray
162         A 2-D array where 's[k]' is the open-loop state at time step 'k',
163         for 'k=0, 1, ..., N-1'.
164     u: numpy.ndarray
165         A 2-D array where 'u[k]' is the open-loop state at time step 'k',
166         for 'k=0, 1, ..., N-1'.
167     J: float
168         The SCP sub-problem cost.
169     """
170     A, B, c = affinize(f, s_prev[:-1], u_prev)
171     A, B, c = np.array(A), np.array(B), np.array(c)
172     n = Q.shape[0]
173     m = R.shape[0]
174     s_cvx = cvx.Variable((N + 1, n))
175     u_cvx = cvx.Variable((N, m))
176
177     # PART (c) #####
178     # INSTRUCTIONS: Construct the convex SCP sub-problem.

```

```

179 objective = cvx.quad_form(s_cvx[N] - s_goal, P)
180 constraints = [s_cvx[0] == s0]
181 for k in range(N):
182     objective += cvx.quad_form(s_cvx[k] - s_goal, Q)
183     objective += cvx.quad_form(u_cvx[k], R)
184     constraints += [
185         s_cvx[k + 1] == A[k] @ s_cvx[k] + B[k] @ u_cvx[k] + c[k],
186         cvx.abs(u_cvx[k, 0]) <= u_max,
187         cvx.norm_inf(s_cvx[k] - s_prev[k]) <= ρ,
188         cvx.norm_inf(u_cvx[k] - u_prev[k]) <= ρ,
189     ]
190 constraints.append(cvx.norm_inf(s_cvx[N] - s_prev[N]) <= ρ)
191 # END PART (c) #####
192
193 prob = cvx.Problem(cvx.Minimize(objective), constraints)
194 prob.solve()
195 if prob.status != "optimal":
196     raise RuntimeError("SCP_solve_failed_Problem_status:" + prob.status)
197 s = s_cvx.value
198 u = u_cvx.value
199 J = prob.objective.value
200 return s, u, J
201
202
203 def cartpole(s, u):
204     """Compute the cart-pole state derivative."""
205     mp = 1.0 # pendulum mass
206     mc = 4.0 # cart mass
207     L = 1.0 # pendulum length
208     g = 9.81 # gravitational acceleration
209
210     x, θ, dx, dθ = s
211     sinθ, cosθ = jnp.sin(θ), jnp.cos(θ)
212     h = mc + mp * (sinθ**2)
213     ds = jnp.array(
214         [
215             dx,
216             dθ,
217             (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
218             -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
219             / (h * L),
220         ]
221     )
222     return ds
223
224
225 def discretize(f, dt):
226     """Discretize continuous-time dynamics f via Runge-Kutta integration."""
227
228     def integrator(s, u, dt=dt):
229         k1 = dt * f(s, u)
230         k2 = dt * f(s + k1 / 2, u)
231         k3 = dt * f(s + k2 / 2, u)
232         k4 = dt * f(s + k3, u)
233         return s + (k1 + 2 * k2 + 2 * k3 + k4) / 6
234
235     return integrator
236
237
238 # Define constants
239 n = 4 # state dimension
240 m = 1 # control dimension
241 s_goal = np.array([0, np.pi, 0, 0]) # desired upright pendulum state

```

```

242 s0 = np.array([0, 0, 0, 0]) # initial downright pendulum state
243 dt = 0.1 # discrete time resolution
244 T = 10.0 # total simulation time
245 P = 1e3 * np.eye(n) # terminal state cost matrix
246 Q = np.diag([1e-2, 1.0, 1e-3, 1e-3]) # state cost matrix
247 R = 1e-3 * np.eye(m) # control cost matrix
248 ρ = 1.0 # trust region parameter
249 u_max = 8.0 # control effort bound
250 eps = 5e-1 # convergence tolerance
251 max_iters = 100 # maximum number of SCP iterations
252 animate = True # flag for animation
253
254 # Initialize the discrete-time dynamics
255 fd = jax.jit(discretize(cartpole, dt))
256
257 # Solve the swing-up problem with SCP
258 t = np.arange(0.0, T + dt, dt)
259 N = t.size - 1
260 s, u, J = solve_swingup_scp(fd, s0, s_goal, N, P, Q, R, u_max, ρ, eps, max_iters)
261
262 # Simulate open-loop control
263 for k in range(N):
264     s[k + 1] = fd(s[k], u[k])
265
266 # Plot state and control trajectories
267 fig, ax = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
268 plt.subplots_adjust(wspace=0.45)
269 labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
270 labels_u = (r"$u(t)$",)
271 for i in range(n):
272     ax[i].plot(t, s[:, i])
273     ax[i].axhline(s_goal[i], linestyle="--", color="tab:orange")
274     ax[i].set_xlabel(r"$t$")
275     ax[i].set_ylabel(labels_s[i])
276 for i in range(m):
277     ax[n + i].plot(t[:-1], u[:, i])
278     ax[n + i].axhline(u_max, linestyle="--", color="tab:orange")
279     ax[n + i].axhline(-u_max, linestyle="--", color="tab:orange")
280     ax[n + i].set_xlabel(r"$t$")
281     ax[n + i].set_ylabel(labels_u[i])
282 plt.savefig(FIG_DIR / "cartpole_swingup_constrained.png", bbox_inches="tight")
283
284 # Plot cost history over SCP iterations
285 fig, ax = plt.subplots(1, 1, dpi=150, figsize=(8, 5))
286 ax.semilogy(J)
287 ax.set_xlabel(r"SCP_iteration_{$i$}")
288 ax.set_ylabel(r"SCP_cost_{$J(\bar{x}^{(i)}, \bar{u}^{(i)})}$")
289 plt.savefig(FIG_DIR / "cartpole_swingup_constrained_cost.png", bbox_inches="tight")
290 # plt.show()
291
292 # Animate the solution
293 if animate:
294     fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
295     ani.save(FIG_DIR / "cartpole_swingup_constrained.mp4", writer="ffmpeg")
296     # plt.show()

```